



COMPUTER SECURITY

Lecture 4: Programs and OS Security

Prof. Dr. Noha Hikal

**Information Technology Dept.
Vice dean for Community Services and Environmental Development**

OUTLINES

- programming errors with security implications:
buffer overflows, incomplete access control
- malicious code:
viruses, worms, trojan horses
- program development controls against malicious code and vulnerabilities:
software engineering principles and practices
- controls to protect against program flaws in execution:
operating system support and administrative controls

Introduction

Protecting programs is at the heart of computer security because they constitute so much of a computing system s.a. (OS, Device drivers, Network infrastructure, DBMS, ... etc)

Two important questions:

1. How do we keep programs free from flaws?
2. How do we protect computing resources against programs that contain flaws?

Bug, Error, Fault, and Failure

Bug in software is a term that can mean many different things depending on context. For example, it can be a mistake in interpreting a requirement or a syntax error in a piece of code.

Error is a human mistake in performing some software activity that may lead to a **fault** in a computer program.

A **fault** may cause a **failure** (which is a departure from the system's required behavior).

a fault is an inside view of the system, seen by the developers, whereas a failure is an outside view seen by the user.

Program security flaws

Unfortunately, we do not have techniques to eliminate or address all program security flaws because:

Program controls apply at the level of the individual program and programmer. When we test a system, we try to make sure that the functionality prescribed in the requirements is implemented in the code (i.e we take a "should do" checklist and verify). However, security is also about preventing certain actions: a "shouldn't do" list.

Programming and software engineering techniques change and evolve far more rapidly than do computer security techniques.

Types of Flaws

Intentional

- Malicious
- Non-malicious

Inadvertent (Unintentional)

- validation error.
- domain error: controlled access to data.
- serialization and aliasing: program flow order.
- inadequate identification and authentication.
- boundary condition violation.
- other exploitable logic errors.

Non-malicious Program Errors

Buffer Overflows

All program and data elements are in memory during execution, sharing space with the operating system, other code, and resident routines. Therefore, the effect of the overflows data depends on where it is go in the memory; It may affect the user data, user code, system data, or system code.

Non-malicious Program Errors (cont.)

Incomplete Mediation

Attackers are exploiting it to cause security problems. eg: changing the price value in a URL.

`http://www.things.com/order.asp?custID=101&part=555A&qy=20&price=10&ship=boat&shipcost=5&total=205`

`http://www.things.com/order.asp?custID=101&part=555A&qy=20&price=10&ship=boat&shipcost=5&total=25`

Non-malicious Program Errors (cont.)

Time-of-Check to Time-of-Use Errors

A delay between the time the access was checked and the time the result of the check was used, a change occurred, invalidating the result of the check.

Example: Changing the file name that checked for a deletion access.

Malicious Code

- By themselves, programs are seldom security threats.
- Most users don't know which programs in addition to their programs are executed or modified, and which files are changed because they usually do not see computer data directly, **malicious people** can make programs serve as vehicles to access and change data and other programs.
- When a user install a SW package, or a plug-in from the Internet, or download an application s.a. a Java applet or an ActiveX control while viewing a web site; a lot of programs and data are transferred and a lot of modifications may be made to your existing files, all occurring without your explicit consent or knowledge.
- → **We should worry about malicious code.**

Malicious Code (cont.)

Malicious code can do anything like any other program (writing data, stopping a running program, erasing a file, ...etc).

Malicious code can do nothing at all right now; it can be planted to lie dormant, undetected, until some event triggers the code to act s.a.:

- A time or date trigger
- An interval (eg. after 30 minutes)
- An event (eg. when a particular program is executed)
- A condition (eg. when communication occurs on a NW)
- A count (eg. the fifth time something happens)
- A random situatio
- etc.

OR some combination of these

OR different things each time

Malicious code runs under the user's authority, but without the user's permission or even knowledge.

Kinds of Malicious Code

Virus is a program that can replicate itself and pass on malicious code to other non-malicious programs by modifying them.

- **Transient**: runs when its attached program executes and terminates when its attached program ends.
- **Resident**: locates itself in memory so that it can remain active even after its attached program ends.

Trojan Horse is malicious code that, in addition to its primary effect, has a second, non-obvious malicious effect.

Kinds of Malicious Code (cont.)

Logic Bomb is a class of malicious code that "detonates" or goes off when a specified condition occurs.

Trapdoor/backdoor is a feature in a program by which someone can access the program other than by the obvious, direct call, perhaps with special privileges.

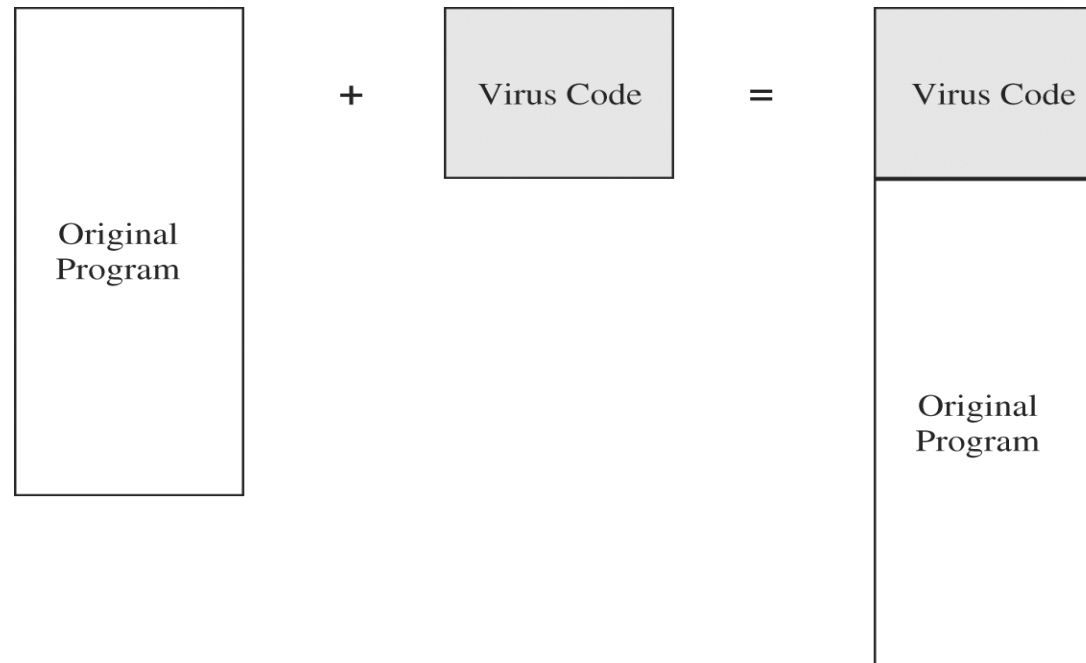
worm is a program that spreads copies of itself through a network.

- A worm operates through networks, and a virus can spread through any medium.
- A worm spreads copies of itself as a stand-alone program, whereas the virus spreads copies of itself as a program that attaches to or embeds in other programs.

Virus	Attaches itself to program and propagates copies of itself to other programs
Trojan horse	Contains unexpected, additional functionality
Logicbomb	Triggers action when condition occurs
Time bomb	bombTriggers action when specified time occursTrapdoorAllows
Trapdoor	Allows unauthorized access to functionality
Worm	Propagates copies of itself through a network
Rabbit	Replicates itself without limit to exhaust resources

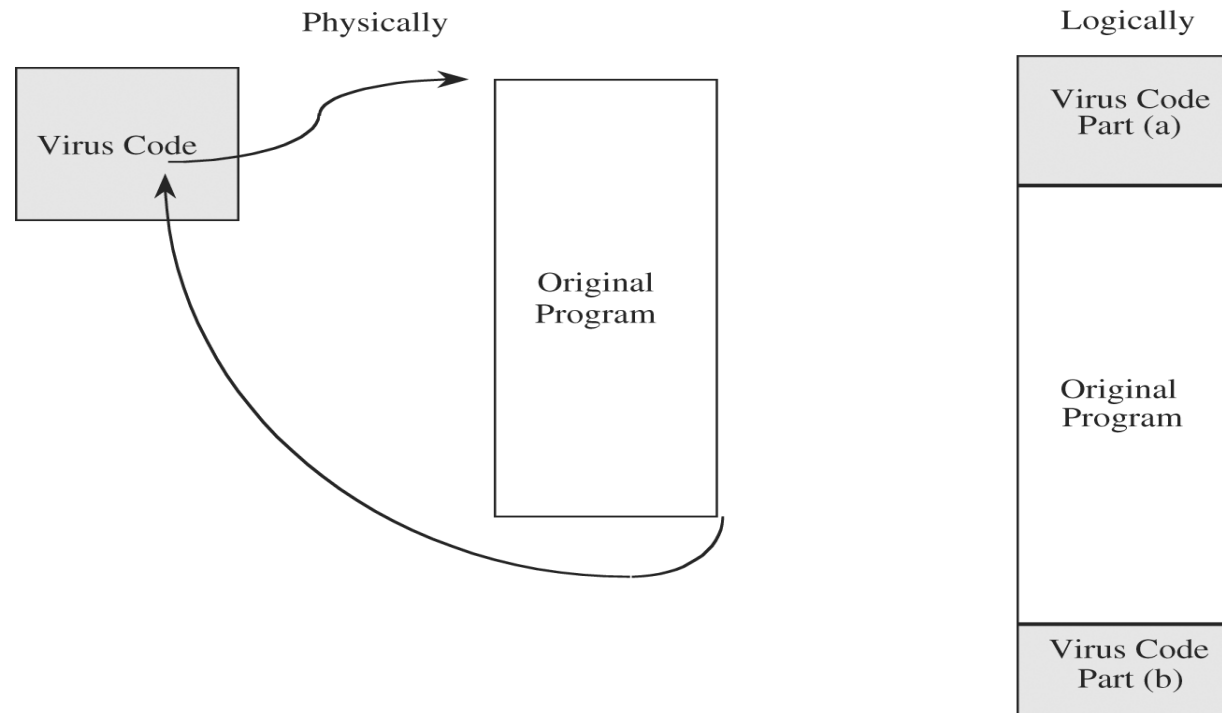
How Viruses Attach?

Appended Viruses: A program virus attaches itself to a program; then, whenever the program is run, the virus is activated. This kind of attachment is usually easy to program.



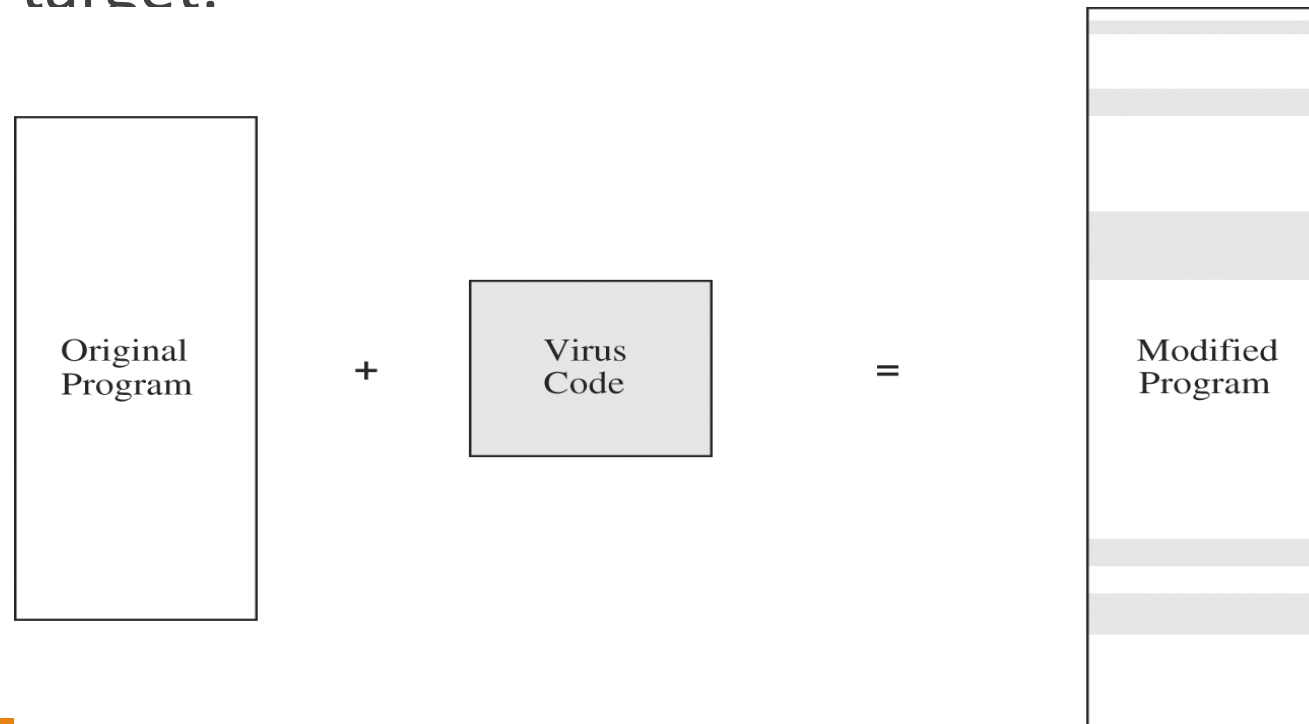
How Viruses Attach (cont.)

Viruses That Surround a Program: A virus that runs the original program but has control before and after its execution.



How Viruses Attach (cont.)

Integrated Viruses and Replacements: A virus replaces some of its target, integrating itself into the original code of the target.



Document Viruses

Document virus is implemented within a formatted document, such as a written document, a database, a slide presentation, a picture, or a spreadsheet.

These documents are highly structured files that contain both data and commands. The commands are part of a rich programming language, including macros, variables and procedures, file accesses, and even system calls.

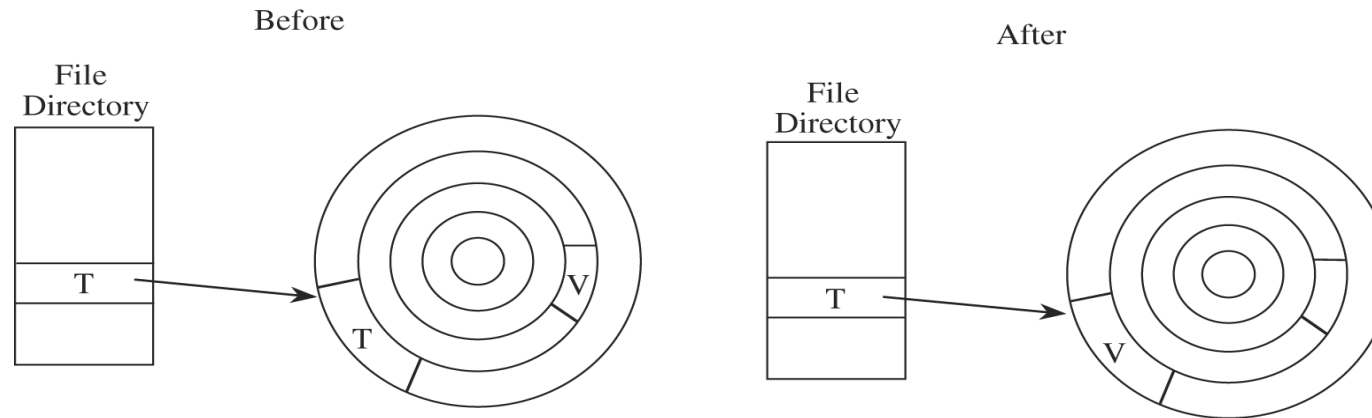
Document Viruses (cont.)

The writer of a document virus uses any of the features of the programming language to perform malicious actions.

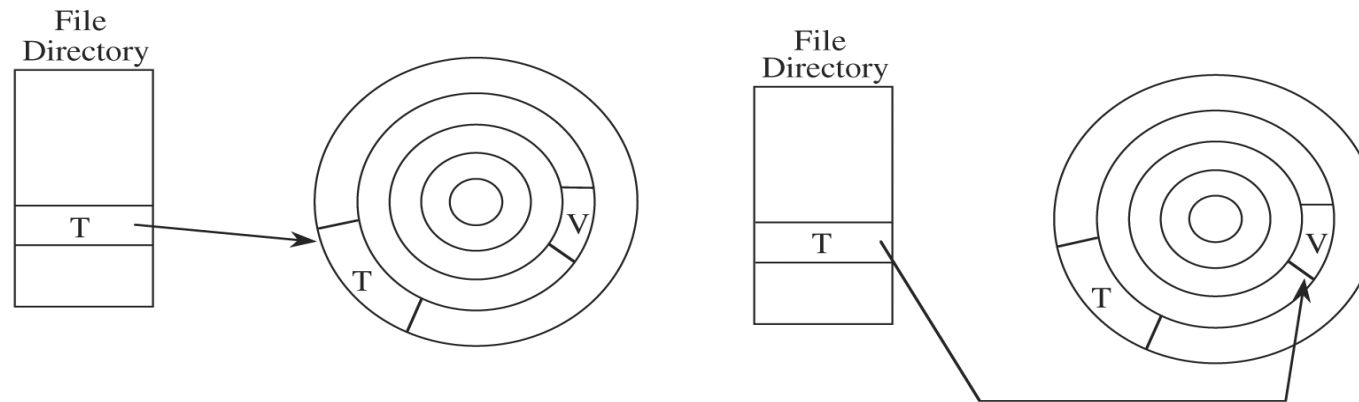
The ordinary user usually sees only the content of the document (its text or data), so the virus writer simply includes the virus in the commands part of the document, as in the integrated program virus.

How Viruses Gain Control

The virus (V) has to be invoked instead of the target (T).



(a) Overwriting T



(b) Changing Pointers

Homes for Viruses

The virus writer may find these qualities appealing in a virus:

It is hard to detect.

It is not easily destroyed or deactivated.

It spreads infection widely.

It can re-infect its home program or other programs.

It is easy to create.

It is machine independent and operating system independent.

***** Few viruses meet all these criteria**

Homes for Viruses (cont.)

One-Time Execution: The majority of viruses today execute only once, spreading their infection and causing their effect in that one execution.

Boot Sector Viruses: Change control on the OS instructions, when a computer is started.

Memory-Resident Viruses: Virus writers also like to attach viruses to resident code because the resident code is activated many times while the machine is running.

Other Homes for Viruses: application programs and libraries.

Virus Signatures

A **virus** cannot be completely invisible. Code must be stored somewhere, and the code must be in memory to execute. Moreover, the virus executes in a particular way, using certain methods to spread. Each of these characteristics yields a telltale pattern, called a **signature**.

Virus scanner looks for the virus signature to detect its existence.

polymorphic virus changes its appearance.

Prevention of Virus Infection

The only way to prevent the infection of a virus is not to receive executable code from an infected source.

Some technique that help:

- Use only commercial software acquired from reliable, well-established vendors.
- Test all new software on an isolated computer.
- Open attachments only when you know them to be safe.
- Make a recoverable system image and store it safely.
- Make and retain backup copies of executable system files.
- Use virus detectors (often called virus scanners) regularly and update them daily.

Controls Against Program Threats

Developmental Controls

Modularity, Encapsulation, and Information Hiding.

Peer Reviews.

Independent Testing.

Good design.

Configuration Management.

Proofs of Program Correctness.

Modularity, Encapsulation, and Information Hiding

Modularization is the process of dividing a task into subtasks on a functional and logical basis.

Advantages: Maintenance, Reuse, Understandability, Correctness, Testing.

Characteristics of a Module: Unity, Smallness, Simplicity, Independence.

Peer Reviews

Each team member has a clear design document.

Team members review each others' code.

All team members recognize that the product belongs to the group.

Independent Testing

To certify the correctness of a program, not necessarily to assign blame for errors

Independent test teams are more effective to test a program than the programmer

Independent testing increases the likelihood that a test will expose the effect of a hidden feature

Configuration Management

To guard against the inadvertent loss of a version of a program.

To manage the parallel development of several similar versions of one program.

To provide facilities for controlled sharing of modules that combine to form one system.

Security Advantages:

- Protects against unintentional threats.
- Guards against malicious ones.

Proofs of Program Correctness

Program verification is a technique to demonstrate formally the “correctness” of a specific program.

Program statements are translated into logical statements about the contribution of that statement to the logical flow of the program.

Proofs of Program Correctness (cont.)

Program correctness proofs are hindered by:

- Program translation is error prone.
- The logical engines are slow.
- Proofs of correctness have not been consistently and successfully applied to large production systems.

THANK YOU

